

# Liquidity Delivery Audit Report



# Table of Contents

---

|                                                                                                     |           |
|-----------------------------------------------------------------------------------------------------|-----------|
| <b>1. Executive Summary</b>                                                                         | <b>4</b>  |
| About M0 Liquidity Delivery                                                                         | 4         |
| Audit Summary                                                                                       | 4         |
| Overall Security Posture                                                                            | 4         |
| After Fix Review Summary                                                                            | 4         |
| Before Fix Review Summary                                                                           | 4         |
| Launch Recommendations                                                                              | 5         |
| Audit Scope                                                                                         | 5         |
| <b>2. Assumptions and Considerations</b>                                                            | <b>6</b>  |
| Audit Assumptions                                                                                   | 6         |
| Centralization Risks                                                                                | 6         |
| Trust Assumptions                                                                                   | 6         |
| <b>3. Severity Definitions</b>                                                                      | <b>7</b>  |
| Impact                                                                                              | 7         |
| Likelihood                                                                                          | 7         |
| Severity Classification Matrix                                                                      | 7         |
| <b>4. Findings</b>                                                                                  | <b>9</b>  |
| L01: Ensure protection against ReportFill replay                                                    | 9         |
| L02: Ensure amount_in_to_release and amount_out_filled are of correct proportion in ReportOrderFill | 10        |
| <b>5. Enhancement Opportunities</b>                                                                 | <b>11</b> |
| E01: Enable solvers to specify custom refundAddress for cross-chain message fee refunds             | 11        |
| E02: Return messageId from _fillOrder to enable better tracking and debugging of cross-chain fills  | 12        |
| E03: Add validation to prevent zero address for originRecipient in fill operations                  | 13        |
| E04: Ensure token_in != token_out in NativeOrder creation                                           | 14        |
| <b>About Us</b>                                                                                     | <b>15</b> |
| About Adevar Labs                                                                                   | 15        |

|                        |    |
|------------------------|----|
| Audit Methodology      | 15 |
| Confidentiality Notice | 16 |
| Legal Disclaimer       | 16 |

# 1. Executive Summary

## About M0 Liquidity Delivery

M0 provides a shared liquidity delivery system that ensures smooth movement between M0-powered stablecoins and major existing stablecoins (USDC, USDT, etc.). It ensures that anyone using an M0 extension can easily convert into and out of it from other stablecoins across multiple chains. Core components include: onchain order contracts (Solana + EVM), a Portal bridge for reporting fills/cancels back to the origin, and solver/relayer flows to execute fills and propagate reports.

This audit covers the onchain order contracts on Solana and EVM; off-chain components and bridge infrastructure are out of scope.

## Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

| Risk Level | Count | Fixed | Acknowledged |
|------------|-------|-------|--------------|
| Critical   | 0     | –     | –            |
| High       | 0     | –     | –            |
| Medium     | 0     | –     | –            |
| Low        | 2     | 1     | 1            |

## Overall Security Posture

Codebase is solid: no exploitable vulnerabilities found; two low-severity issues and four enhancement opportunities identified. Core checks (order IDs, deadlines, solver auth, balance changes) are consistently enforced across Solana and EVM.

## After Fix Review Summary

All enhancement recommendations were implemented. One low-severity issue was remediated; the remaining low-severity issue is acknowledged because the Portal reporting flow provides replay protection for fill reports.

## Before Fix Review Summary

Strengths: consistent state gating, math bounds enforced by caps, fee-on-transfer protections on local transfers. Low issues: enforce price ratio on reported fills; add replay protection for fill reports. Enhancements: validate non-zero `originRecipient`, allow custom refund address and surface `messageld` for cross-chain fills, and optionally disallow `token_in == token_out` for native orders.

## Launch Recommendations

1. Address the two low severity findings. Add price-ratio enforcement and replay/idempotency in `ReportOrderFill/reportFill`.
2. Harden fill UX: reject zero `originRecipient`, optionally reject `token_in == token_out` for native orders.
3. Improve cross-chain operability: allow custom `refundAddress` and emit/return bridge `messageId` for fills.

## Audit Scope

- **Repository:** <https://github.com/m0-foundation/liquidity-delivery>
- **Before Fix Review Commit Hash:** `53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566`
- **After Fix Review Commit Hash:** `17d21cda5f5131e87d669efe504bd8fe0df8de6e`
- **Files/Modules in Scope:** `svm/src`, `evm/programs/order_book/src`

## 2. Assumptions and Considerations

---

### Audit Assumptions

**I. Portal authority honesty and delivery:** The portal authority behaves correctly, forwards **ReportFill/ReportCancel** messages from destination to origin, and does not replay them (reports are not idempotent by default).

**V. Recipient smart contracts (cross-chain):** If a cross-chain recipient is a smart contract, it can cancel before deadline when needed; otherwise an infinite deadline could strand funds.

### Centralization Risks

- Admin/pauser roles can pause/unpause and update destinations.
- Portal authority controls cross-chain report submission; misuse can block or distort fills/cancels.
- Bridge adapter choice and Portal dependency centralize cross-chain delivery.

### Trust Assumptions

- Portal bridge and relayers deliver messages faithfully.
- Off-chain solvers/relayers follow expected workflows (e.g., include correct accounts, send reports).

## 3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

*Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.*

### Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

### Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

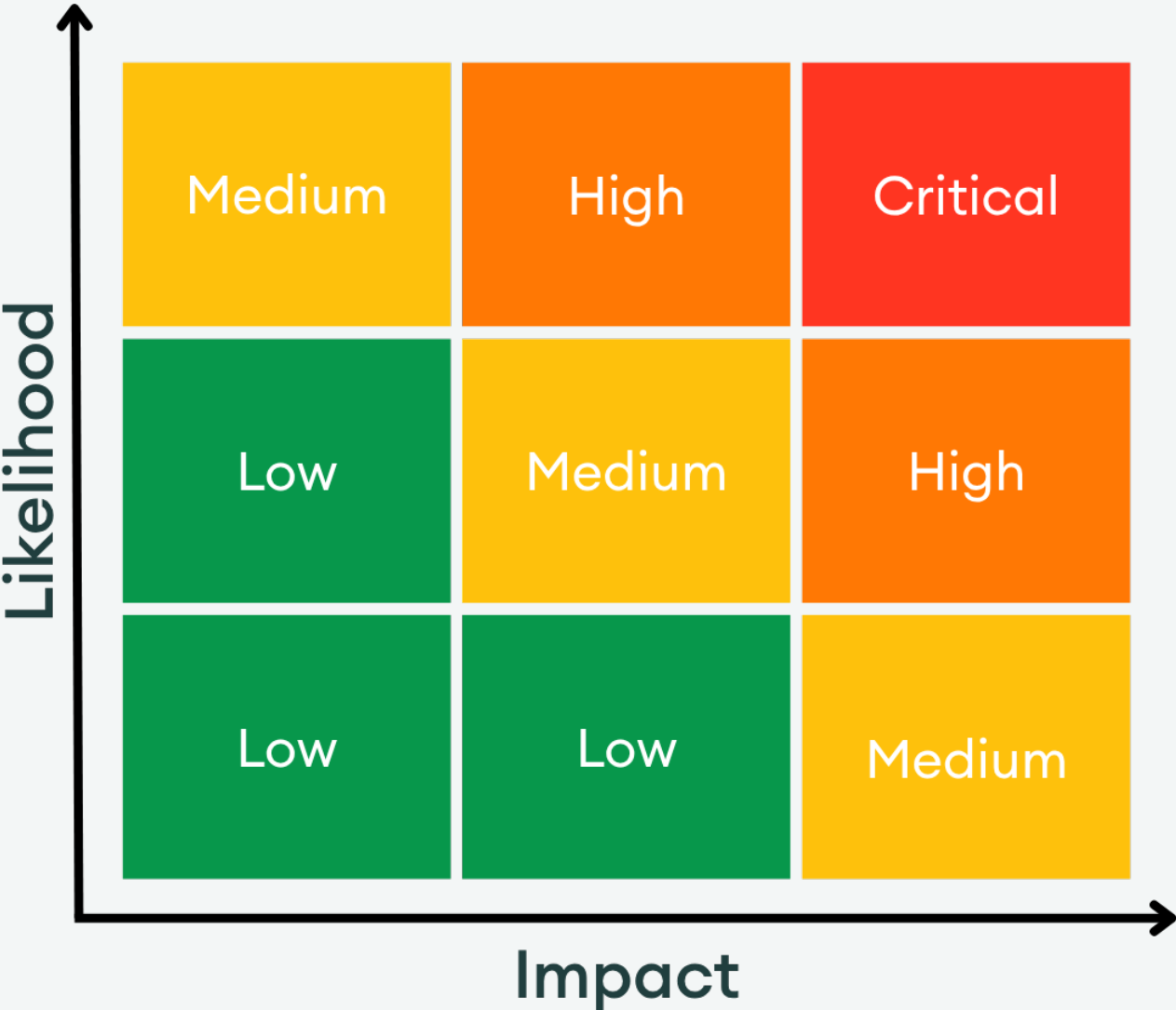
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

### Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix



## 4. Findings

### L01: Ensure protection against ReportFill replay

Status:

Acknowledged

Impact:



Likelihood:



Severity:

Low

**Location:** [https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order\\_book/src/instructions/fill.rs#L627-L631](https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order_book/src/instructions/fill.rs#L627-L631)

>\_ fill.rs

RUST

```
625: impl ReportOrderFill<'_> {
626:     fn validate(&self, fill_report: &FillReport) -> Result<()> {
627:         // Validate the order type is native
628:         require!(
629:             self.order.order_type == OrderType::Native,
630:             OrderBookError::InvalidOrderType
631:         );
632:
633:         let status = &self.order.data.status;
```

#### Description:

`ReportOrderFill` accepts incremental fill reports and simply adds `amount_out_filled` / `amount_in_to_release` to the order. The same report can be replayed and counted again (as long as totals stay under `amount_out/amount_in`), letting a single fill message drain more funds than the actual fill.

#### Recommendation:

Make fill reports idempotent: track a per-order report nonce/id/hash. Apply the same protection to the EVM `reportFill`. It would prevent replayed/duplicate fill reports from over-releasing `amount_in`, ensuring each fill is applied exactly once even if the portal authority resubmits the same message.

#### Developer Response:

The replay attacks are handled by the portal and it would require a bit of a refactor to add this in the order book. We don't anticipate using this contract with anything but the portal so I don't think it will practically become an issue.

## L02: Ensure `amount_in_to_release` and `amount_out_filled` are of correct proportion in `ReportOrderFill`

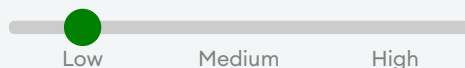
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

**Location:** [https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order\\_book/src/instructions/fill.rs#L654-L656](https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order_book/src/instructions/fill.rs#L654-L656)

> `fill.rs`

RUST

```
652:     let order = &mut ctx.accounts.order.data;
653:
654:     // Update the filled amounts on the order
655:     order.amount_in_released += fill_report.amount_in_to_release;
656:     order.amount_out_filled += fill_report.amount_out_filled;
657:
658:     // Validate the filled amounts do not exceed the order amounts
```

### Description:

`ReportOrderFill` trusts `fill_report.amount_in_to_release` relative to `amount_out_filled` without checking the order's price. A compromised portal authority could unlock most of `amount_in` for a tiny reported `amount_out_filled`, violating order terms.

### Recommendation:

In `ReportOrderFill`, recompute the expected `amount_in_to_release` from stored order terms. Apply equivalent validation on the EVM `reportFill`. It would protect origin-chain funds from over-release due to bad fill reports and preserves the agreed exchange rate even if the portal authority misbehaves.

### Developer Response:

Fixed.

### Mitigation Analysis:

Fixed according to the recommendation.

## 5. Enhancement Opportunities

### E01: Enable solvers to specify custom `refundAddress` for cross-chain message fee refunds

**Location:** <https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/evm/src/OrderBook.sol#L573>

```
> _ OrderBook.sol SOLIDITY
571:         orderData_.originChainId, // destinationChainId (of this message)
572:         report_, // report
573:         msg.sender.toBytes32(), // refundAddress
574:         bridgeAdapterArgs_
575:     )
```

#### Description:

Currently, the `refundAddress` parameter passed to `IPortalV2Like.sendFillReport` is hardcoded to `msg.sender.toBytes32()`, meaning excess native gas is always refunded to the transaction sender.

When solvers operate through relayers, smart contract wallets, or delegated execution, they may need refunds routed to a different address.

#### Potential Benefit:

Solvers can route refunds to their preferred address.

#### Recommendation:

Add an optional `refundAddress` field to `FillParams` struct.

#### Developer Response:

Fixed.

## E02: Return `msg.msgId` from `_fillOrder` to enable better tracking and debugging of cross-chain fills

**Location:** <https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/evm/src/OrderBook.sol#L570>

```
> _ OrderBook.sol SOLIDITY
568:         // to the portal for crosschain message fee
569:         bridgeAdapter_ == address(0)
570:         ? IPortalV2Like(portal).sendFillReport{ value: msg.value }(
571:             orderData_.originChainId, // destinationChainId (of this message)
572:             report_, // report
```

### Description:

The `IPortalV2Like.sendFillReport` function returns a `bytes32 msgId` that uniquely identifies the cross-chain message, but `_fillOrder` does not capture or return this value.

### Potential Benefit:

Better tracking and debugging: solvers can observe more complete info from `OrderFilled` event if the `msgId` will be included there.

### Recommendation:

Update `fillOrder` to return `msgId` and forward the return value into the `OrderFilled` event emitted.

### Developer Response:

Fixed.

## E03: Add validation to prevent zero address for originRecipient in fill operations

**Location:** <https://github.com/m0-foundation/liquidity-delivery/blob/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/evm/src/OrderBook.sol#L559-L565>

| > _ OrderBook.sol | SOLIDITY                                                         |
|-------------------|------------------------------------------------------------------|
| 557:              | // we need to send a message back to the origin chain to release |
| 558:              | // the corresponding amount of tokenIn to the solver's recipient |
| 559:              | FillReport memory report_ = FillReport({                         |
| 560:              | orderId: orderId_,                                               |
| 561:              | originRecipient: fillerParams_.originRecipient,                  |
| 562:              | amountOutFilled: amountOutToFill_,                               |
| 563:              | amountInToRelease: amountInToRelease_,                           |
| 564:              | tokenIn: orderData_.tokenIn                                      |
| 565:              | });                                                              |
| 566:              |                                                                  |
| 567:              | // Send fill report to the origin chain and pass along msg.value |

### Description:

Currently, the `_fillOrder` function does not validate that `fillerParams_.originRecipient` is not `bytes32(0)`. This could lead to funds being sent to the zero address in local fills or cause `reportFill` to revert on the source chain during cross-chain fills

### Potential Benefit:

Validate the `originRecipient` to prevents cross-chain failures.

### Recommendation:

Add validation at the beginning of `_fillOrder` function, similar to the recipient validation in `_openOrder`:

| SOLIDITY                                                                                 |
|------------------------------------------------------------------------------------------|
| <code>if (fillerParams_.originRecipient == bytes32(0)) revert InvalidRecipient();</code> |

### Developer Response:

Fixed.



## E04: Ensure `token_in != token_out` in `NativeOrder` creation

**Location:** [https://github.com/m0-foundation/liquidity-delivery/tree/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order\\_book/src/instructions/open.rs#L114-L125](https://github.com/m0-foundation/liquidity-delivery/tree/53510e6dbf5ca5ae2fae9fd3f79c7f4f75210566/svm/programs/order_book/src/instructions/open.rs#L114-L125)

```
>_ open.rs RUST
112: impl OpenOrder<'_> {
113:     fn validate(&self, params: &OrderParams) -> Result<()> {
114:         // Validate the destination
115:         // If the destination chain is not the current chain, ensure the destination i
           s supported
116:         if params.dest_chain_id != self.global_account.chain_id {
117:             let destination_account = self
118:                 .destination_account
119:                 .as_ref()
120:                 .ok_or(OrderBookError::DestinationNotSupported)?;
121:             require!(
122:                 destination_account.is_supported,
123:                 OrderBookError::DestinationNotSupported
124:             );
125:         }
126:
127:         // Validate params
```

### Description:

Native orders currently allow `token_in == token_out`, which makes the order effectively a no-op and can lead to confusing/failed fills depending on how accounts are supplied. It's a low-severity/UX issue rather than a loss-of-funds vector.

### Potential Benefit:

Avoid users creating orders that can't meaningfully execute and reduce unexpected fill failures.

### Recommendation:

Add a validation in `OpenOrder` rejecting native orders where `token_in` equals `token_out`.

### Developer Response:

Fixed.

# About Us

---

## About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

## Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

### 1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

### 2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)

- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

### 3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

### 4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

## Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

## Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in

connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.